

LAB 2 · 레지스터

작성일 2026. 09. 13. · 이해리

빈 템플릿 → 직접 작성 → Icarus 시뮬레이션 → Vivado 구현

목차

프로젝트 시작

RTL·TB·XDC 작성

VS Code 실행과 파형

실험 전 레포트

Vivado 구현

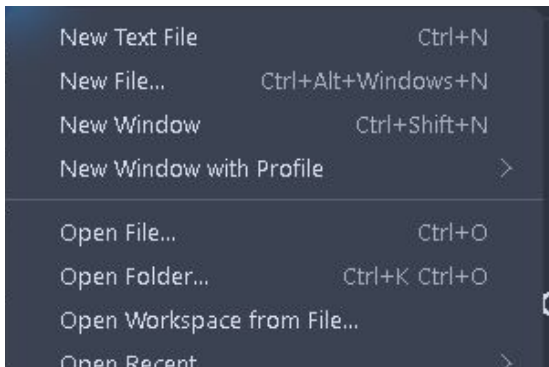
실험 후 레포트

v2.0.1 템플릿을 새 이름으로 clone

Windows PowerShell에서 한 줄씩 실행한다.

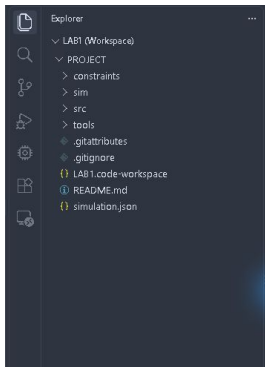
```
git clone --branch v2.0.1 `
https://github.com/Glaysia/fpga-lab-template.git `
lab2_03_register
cd lab2_03_register
git switch -c main
code LAB1.code-workspace
```

새 창과 워크스페이스



Ctrl+Shift+N으로 새 창을 연다. File → Open Workspace from File... → 자신의 실습 폴더 → LAB1.code-workspace. 화면은 v2.0.0 빈 템플릿의 공통 예시다. v2.0.1에서도 여는 방법은 같고 .slang 설정이 추가되어 있다.

빈 템플릿 확인



clone 직후에는 정답 코드가 없다. Explorer의 `src·sim·constraints`를 펼쳐 빈 파일을 확인하고 뒤의 내용을 작성한다. 화면은 v2.0.0 빈 템플릿의 공통 예시다. v2.0.1에서도 여는 방법은 같고 `.slang` 설정이 추가되어 있다.

workspace와 추천 확장

1. lab2_03_register 폴더에서 LAB1.code-workspace 선택 → Open.
2. Explorer에 PROJECT 하나와 src.sim.constraints.tools가 보이는지 확인.
3. Extensions → @recommended를 검색.
4. slang.VaporView.vscode-pdf의 Install을 누른다.
5. Python과 Icarus 실행 파일은 별도로 설치되어 있어야 한다.

v2.0.1 편집기와 시뮬레이터

1. slang 확장·서버는 0.3.0을 기준으로 확인한다. 추천 목록은 버전을 고정하지 않는다.
2. 템플릿의 .slang/server.json은 src·sim 탐색용이다. 새 RTL은 simulation.json에도 직접 등록한다.
3. Ctrl+Shift+P → slang: Show Output. 서버 오류가 있으면 README의 버전 확인·재시작 절차를 따른다.
4. slang.path에 다른 사람의 설치 경로를 복사하지 않는다. 직접 지정할 때는 User 설정에 자신의 경로를 쓴다.
5. VS Code 02 Simulate는 Icarus, Vivado Run Behavioral Simulation은 XSim이다.

템플릿 도구 설치·오류 해결 안내

시뮬레이션 도구 확인

1. Terminal → Run Task... → 01 Check tools.
2. Git·Python·iverilog·vvp의 버전이 출력되는지 확인한다.
3. 도구를 찾지 못하면 설치 경로를 PATH에 추가하고 VS Code를 다시 연다.

```
git --version  
python --version  
iverilog -V  
vvp -V
```

빈 파일에서 시작

1. src/design.v를 우클릭 → Rename. 첫 RTL 파일명으로 바꾼다.
2. 추가 RTL은 src를 우클릭 → New File로 만든다.
3. sim/tb_design.v를 이번 TB 파일명으로 바꾼다.
4. constraints/pins.xdc를 이번 XDC 파일명으로 바꾼다.
5. 뒤의 코드 화면을 보며 작성하고 File → Save All.

이번 프로젝트에 작성할 파일

역할	경로
src	src/register_pair.v
src	src/input_frontend.v
src	src/lab2_register.v
sim	sim/tb_register_pair.sv
constraints	constraints/lab2_register.xdc

simulation.json의 세 항목

`sources`: 앞 표의 `src/` 파일 경로를 배열로 넣는다.

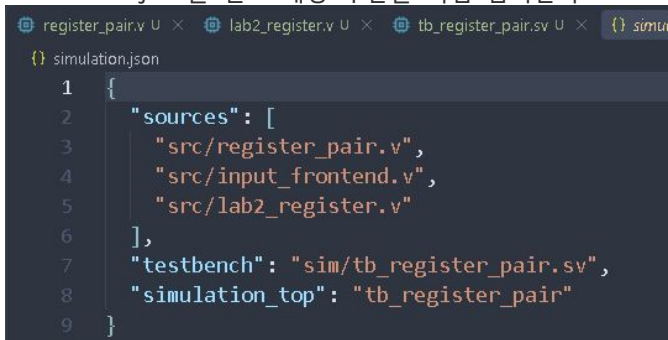
`testbench`: `sim/tb_register_pair.sv`

`simulation_top`: `tb_register_pair`

큰따옴표·대괄호·심표를 확인한다. `module` 이름에는 파일 확장자를 붙이지 않는다.

simulation.json · 1-9행

simulation.json를 열고 해당 구간을 직접 입력한다.



```
1 {  
2   "sources": [  
3     "src/register_pair.v",  
4     "src/input_frontend.v",  
5     "src/lab2_register.v"  
6   ],  
7   "testbench": "sim/tb_register_pair.sv",  
8   "simulation_top": "tb_register_pair"  
9 }
```

저장과 전달은 별개다

stored는 입력을 저장하는 4비트 레지스터, value는 저장값을 전달받는 4비트 레지스터다.

모두 clk 상승 에지에서 갱신된다. rst=1이면 두 레지스터를 먼저 0으로 만든다.

load=1이면 data_in을 stored에 저장한다. transfer=1이면 에지 직전 stored를 value에 전달한다.

한 에지의 다음 상태

rst	load	transfer	다음 stored	다음 value
1	무관	무관	0	0
0	0	0	유지	유지
0	1	0	data_in	유지
0	0	1	유지	이전 stored
0	1	1	data_in	이전 stored

두 if와 nonblocking 대입

두 if는 독립적으로 검사한다. else if로 바꾸면 load=1일 때 transfer를 검사하지 않게 된다.

stored $\leq$ data_in과 value $\leq$ stored의 오른쪽 항은 같은 에지의 이전 상태에서 읽는다.

예: stored=A, data_in=3에서 두 제어기가 1이면 에지 뒤 stored=3, value=A다. 다음 에지에도 둘 다 1이면 value=3이 된다.

register_pair.v · 1-16행

src/register_pair.v를 열고 해당 구간을 직접 입력한다.

```
src > register_pair.v > ...
1 | timescale 1ns/1ps
2 | module register_pair (
3 |     input wire clk, rst, load, transfer,
4 |     input wire [3:0] data_in,
5 |     output reg [3:0] stored, value
6 | );
7 |     always @(posedge clk) begin
8 |         if (rst) begin
9 |             stored <= 4'd0;
10 |            value <= 4'd0;
11 |         end else begin
12 |             if (load) stored <= data_in;
13 |             if (transfer) value <= stored;
14 |         end
15 |     end
16 | endmodule
```

input_frontend.v · 1-16행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
1  timescale 1ns/1ps
2  // Main clock is 1 kHz: STABLE_CYCLES=20 means 20 ms.
3  // Set switches first, then press the button; the bus is not an atomic CDC.
4  module input_frontend #(parameter integer STABLE_CYCLES = 20) (
5      input wire clk, rst, button,
6      input wire [7:0] sw,
7      output wire reset,
8      output reg press,
9      output wire [7:0] switches
10 );
11     (* ASYNC_REG = "TRUE" *) reg [1:0] reset_pipe;
12     (* ASYNC_REG = "TRUE" *) reg button_meta, button_sync;
13     (* ASYNC_REG = "TRUE" *) reg [7:0] sw_meta, sw_sync;
14     localparam integer WIDTH = (STABLE_CYCLES < 2) ? 1 : $clog2(STABLE_CYCLES);
15     reg [WIDTH-1:0] count;
16     reg accepted;
```

input_frontend.v · 17-32행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
17 assign reset = reset_pipe[1];
18 assign switches = sw_sync;
19 // Asynchronous assertion; release follows two main-clock edges.
20 always @(posedge clk or posedge rst) begin
21     if (rst) reset_pipe <= 2'b11;
22     else reset_pipe <= {reset_pipe[0], 1'b0};
23 end
24 always @(posedge clk or posedge reset) begin
25     if (reset) begin
26         button_meta <= 0; button_sync <= 0;
27         sw_meta <= 0; sw_sync <= 0;
28     end else begin
29         button_meta <= button; button_sync <= button_meta;
30         sw_meta <= sw; sw_sync <= sw_meta;
31     end
32 end
```

input_frontend.v · 33–46행

src/input_frontend.v를 열고 해당 구간을 직접 입력한다.

```
33     always @(posedge clk or posedge reset) begin
34         if (reset) begin
35             count <= 0; accepted <= 0; press <= 0;
36         end else begin
37             press <= 0;
38             if (button_sync == accepted) count <= 0;
39             else if (count == STABLE_CYCLES - 1) begin
40                 accepted <= button_sync;
41                 count <= 0;
42                 press <= button_sync;
43             end else count <= count + 1'b1;
44         end
45     end
46 endmodule
```

lab2_register.v · 1-16행

src/lab2_register.v를 열고 해당 구간을 직접 입력한다.

```
src > lab2_register.v > ...
1 | timescale 1ns/1ps
2 | // Combo II-DLD S75. Main clock B6 MUST be set to 1 kHz.
3 | // K4: reset. M8: step button. DIPSW1..8 = sw[7]..sw[0].
4 | module lab2_register #(parameter integer STABLE_CYCLES = 20) (
5 |     input wire clk, rst, button,
6 |     input wire [7:0] sw,
7 |     output wire [7:0] led
8 | );
9 |     wire reset, press;
10 |    wire [7:0] switches;
11 |    input_frontend #(.STABLE_CYCLES(STABLE_CYCLES)) inputs(
12 |        clk, rst, button, sw, reset, press, switches);
13 |    wire [3:0] stored, value;
14 |    register_pair core(clk, reset, press && switches[0],
15 |        press && switches[1], switches[7:4], stored, value);
16 |    assign led = {stored, value};
```

lab2_register.v · 17-17행

src/lab2_register.v를 열고 해당 구간을 직접 입력한다.

```
17  endmodule
```

버튼을 클록으로 쓰지 않는다

input_frontend는 외부 버튼과 스위치를 주 클록에 동기화한다. 버튼은 20회 연속 안정된 샘플을 확인한 뒤 한 클록의 press를 만든다.

보드 주 클록은 1 kHz이므로 20샘플은 20 ms다. 버튼을 누르고 놓는 한 동작으로 한 번 실행한다.

ASYNC_REG 속성은 동기화 레지스터임을 도구에 알린다. 속성 자체가 버튼 디바운스 기능을 만드는 것은 아니다.

보드 입출력 연결

항목	역할
K4 / rst	두 레지스터 초기화
N8 / button	선택한 동작 한 번 실행
sw[0]	load 선택
sw[1]	transfer 선택
sw[7:4]	4비트 data_in
LED[7:4] / LED[3:0]	stored / value

DIP SW1-SW8은 sw[7]-sw[0] 순서다. sw[3:2]는 사용하지 않는다.

테스트벤치의 일곱 검사

검사 시각	입력 / 동작	stored,value
6 ns	reset	0,0
16 ns	A 저장	A,0
26 ns	입력은 3, 저장값 전달	A,A
36 ns	3 저장과 전달 동시	3,A
46 ns	다음 에지에도 동시	3,3
56 ns	입력 F, 두 제어 0	3,3
66 ns	리셋과 두 제어 동시	0,0

왜 에지 뒤 1 ns에 검사할까

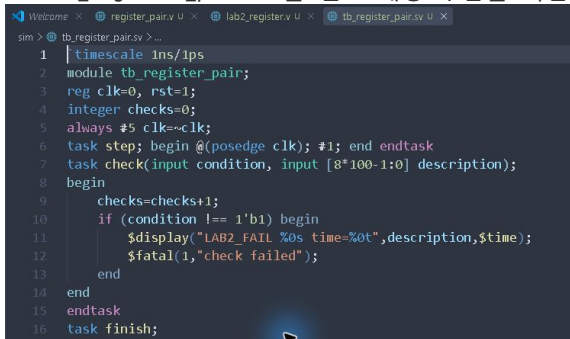
step은 @(posedge clk) 뒤 #1을 기다린다. nonblocking 갱신이 반영된 다음 상태를 비교하기 위해서다.

TB 클록 주기는 10 ns다. 첫 리셋 에지는 5 ns이고 비교는 6 ns에 실행한다.

불일치가 있으면 LAB2_FAIL과 \$fatal로 중단한다. 마지막 비교까지 통과해야 LAB2_PASS를 출력한다.

tb_register_pair.sv · 1-16행

sim/tb_register_pair.sv를 열고 해당 구간을 직접 입력한다.



```
1 | timescale 1ns/1ps
2 | module tb_register_pair;
3 | reg clk=0, rst=1;
4 | integer checks=0;
5 | always #5 clk=~clk;
6 | task step; begin @(posedge clk); #1; end endtask
7 | task check(input condition, input [8*100-1:0] description);
8 | begin
9 |     checks=checks+1;
10 |     if (condition !== 1'b1) begin
11 |         $display("LAB2_FAIL %0s time=%0t",description,$time);
12 |         $fatal(1,"check failed");
13 |     end
14 | end
15 | endtask
16 | task finish;
```

tb_register_pair.sv · 17-32행

sim/tb_register_pair.sv를 열고 해당 구간을 직접 입력한다.

```
17 begin
18     $display("LAB2_PASS register_pair checks=%0d",checks);
19     $finish;
20 end
21 endtask
22 initial begin $dumpfile("wave.vcd"); $dumpvars(0,tb_register_pair); end
23 initial begin #100000; $fatal(1,"watchdog timeout"); end
24 reg load=0, transfer=0;
25 reg [3:0] data_in=0;
26 wire [3:0] stored,value;
27 register_pair dut(clk,rst,load,transfer,data_in,stored,value);
28 initial begin
29     step; check({stored,value}===8'h00,"reset"); rst=0;
30     data_in=4'ha; load=1; step;
31     check({stored,value}===8'ha0,"load does not transfer");
32     load=0; data_in=4'h3; transfer=1; step;
```

tb_register_pair.sv · 33–43행

sim/tb_register_pair.sv를 열고 해당 구간을 직접 입력한다.

```
33     check({stored,value}===8'haa,"transfer stored not live input");
34     load=1; step;
35     check({stored,value}===8'h3a,"simultaneous uses old stored");
36     step; check({stored,value}===8'h33,"next edge transfers new stored");
37     load=0; transfer=0; data_in=4'hf; step;
38     check({stored,value}===8'h33,"both hold");
39     rst=1; load=1; transfer=1; step;
40     check({stored,value}===8'h00,"reset beats both controls");
41     finish;
42 end
43 endmodule
```

XDC 전체를 직접 작성

constraints/lab2_register.xdc를 만들고 다음 세 화면을 모두 입력한다.

lab2_register.v의 포트 이름·비트 번호와 get_ports를 대조한다.

Icarus는 XDC를 사용하지 않는다. 코어 TB 통과만으로 핀 연결과 실제 보드 동작이 검증되는 것은 아니다.

lab2_register.xdc · 1-16행

constraints/lab2_register.xdc를 열고 해당 구간을 직접 입력한다.

```
1  # Combo II-DLD S75 / xc7s75fgga484-1 / main clock 1 kHz.
2  # Pin source and array polarity: evidence/board-pin-provenance.json.
3  set_property PACKAGE_PIN B6 [get_ports {clk}]
4  set_property IOSTANDARD LVCMOS33 [get_ports {clk}]
5  set_property PACKAGE_PIN K4 [get_ports {rst}]
6  set_property IOSTANDARD LVCMOS33 [get_ports {rst}]
7  set_property PACKAGE_PIN N8 [get_ports {button}]
8  set_property IOSTANDARD LVCMOS33 [get_ports {button}]
9  set_property PACKAGE_PIN U4 [get_ports {sw[0]}]
10 set_property IOSTANDARD LVCMOS33 [get_ports {sw[0]}]
11 set_property PACKAGE_PIN V4 [get_ports {sw[1]}]
12 set_property IOSTANDARD LVCMOS33 [get_ports {sw[1]}]
13 set_property PACKAGE_PIN W1 [get_ports {sw[2]}]
14 set_property IOSTANDARD LVCMOS33 [get_ports {sw[2]}]
15 set_property PACKAGE_PIN W4 [get_ports {sw[3]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {sw[3]}]
```

lab2_register.xdc • 17-32행

constraints/lab2_register.xdc를 열고 해당 구간을 직접 입력한다.

```
17 set_property PACKAGE_PIN T1 [get_ports {sw[4]}]
18 set_property IOSTANDARD LVCMOS33 [get_ports {sw[4]}]
19 set_property PACKAGE_PIN U2 [get_ports {sw[5]}]
20 set_property IOSTANDARD LVCMOS33 [get_ports {sw[5]}]
21 set_property PACKAGE_PIN W3 [get_ports {sw[6]}]
22 set_property IOSTANDARD LVCMOS33 [get_ports {sw[6]}]
23 set_property PACKAGE_PIN Y1 [get_ports {sw[7]}]
24 set_property IOSTANDARD LVCMOS33 [get_ports {sw[7]}]
25 set_property PACKAGE_PIN N5 [get_ports {led[0]}]
26 set_property IOSTANDARD LVCMOS33 [get_ports {led[0]}]
27 set_property PACKAGE_PIN M1 [get_ports {led[1]}]
28 set_property IOSTANDARD LVCMOS33 [get_ports {led[1]}]
29 set_property PACKAGE_PIN M3 [get_ports {led[2]}]
30 set_property IOSTANDARD LVCMOS33 [get_ports {led[2]}]
31 set_property PACKAGE_PIN M7 [get_ports {led[3]}]
32 set_property IOSTANDARD LVCMOS33 [get_ports {led[3]}]
```

lab2_register.xdc • 33-42행

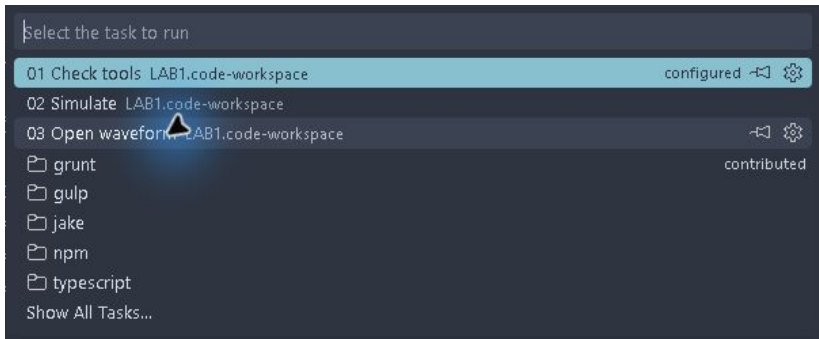
constraints/lab2_register.xdc를 열고 해당 구간을 직접 입력한다.

```
33 set_property PACKAGE_PIN N7 [get_ports {led[4]}]
34 set_property IOSTANDARD LVCMOS33 [get_ports {led[4]}]
35 set_property PACKAGE_PIN M2 [get_ports {led[5]}]
36 set_property IOSTANDARD LVCMOS33 [get_ports {led[5]}]
37 set_property PACKAGE_PIN M4 [get_ports {led[6]}]
38 set_property IOSTANDARD LVCMOS33 [get_ports {led[6]}]
39 set_property PACKAGE_PIN L4 [get_ports {led[7]}]
40 set_property IOSTANDARD LVCMOS33 [get_ports {led[7]}]
41 create_clock -name trainer_1khz -period 1000000.000 [get_ports clk]
42 set_false_path -from [get_ports {rst button sw[*]}]
```

VS Code에서 실행

1. File → Save All로 모든 파일을 저장한다.
2. Terminal → Run Task... → 01 Check tools로 도구를 확인한다.
3. 같은 메뉴에서 02 Simulate를 선택한다. Ctrl+Shift+B도 기본 시뮬레이션 작업을 실행한다.
4. 컴파일 오류는 첫 오류의 파일·행 번호부터 확인한다.

작업 메뉴의 세 번호



같은 템플릿의 공통 작업 메뉴다. 01 도구 확인 → 02 시뮬레이션 → 03 파형 열기 순서로 진행한다.

레지스터 검사 통과

```
Executing task in folder PROJECT: C:/Users/UOS/AppData/Local/Programs/Python/Python312/python.exe tools/lab1.py simulate
C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_03_register\sim\tb_register_pair.sv

RUN C:\msys64\mingw64\bin\vvp.EXE -H C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_03_register\build\sim\run-be
90debc74941b3d320c7e1c430ea\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS register_pair checks=7
C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_03_register\sim\tb_register_pair.sv:19: $finish called at 66000 (
)

시뮬레이션 실행 완료. 진리표 자기검사 파형으로 회로의 정답 여부를 확인하세요.
WAVE C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_03_register\build\sim\wave.vcd
Terminal will be reused by tasks, press any key to close it.
```

LAB2_PASS register_pair checks=7과 66000 ps(66 ns) 종료를 확인한다.
이전 실행의 PASS가 아닌 방금 생성된 로그를 읽는다.

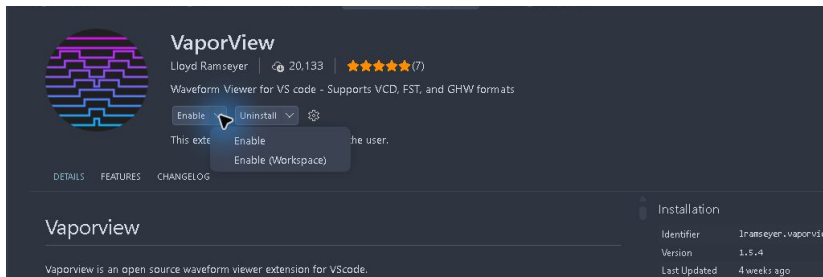
편집기와 실행 도구 문제 구분

WindowsApps·9009 오류가 나면 실제 Python 설치 경로를 확인한다.
LAB1.code-workspace의 windows → command에는 자신의 python.exe 경로를 쓴다.

정적 분석 확장과 Icarus 실행은 별개다. slang 확장과 서버 버전이 맞지 않으면 공식 릴리스의 호환 서버를 설치하고 slang.path를 자신의 경로로 지정한다.

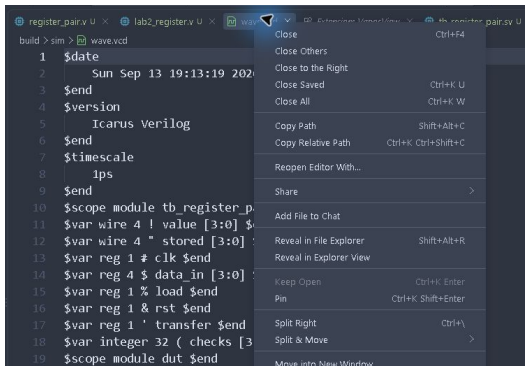
다른 사람의 C:/Users/... 경로를 복사하지 않는다. 환경 수정 내용과 실제 실행 버전을 레포트에 기록한다.

VaporView 활성화



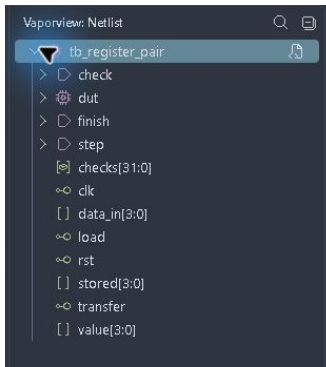
Extensions → @installed vaporview. 꺼져 있다면 Enable 옆 화살표 → Enable (Workspace)를 선택한다.

VCD를 파형 편집기로 열기



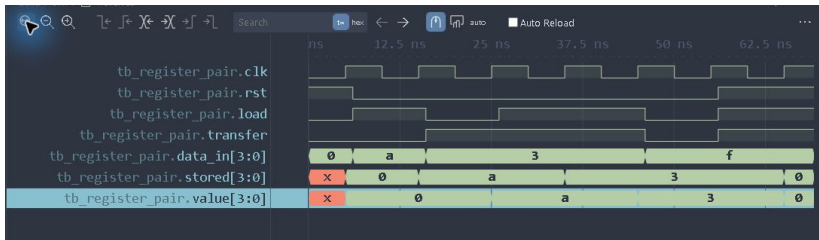
build/sim/wave.vcd를 연다. 텍스트로 열리면 탭 우클릭 → Reopen Editor With... → VaporView를 선택한다.

일곱 신호 추가



Netlist View에서 tb_register_pair를 펼친다. clk·rst·load·transfer·data_in·stored·value를 차례로 더블클릭한다.

전체 파형과 시간 단위



Zoom Fit으로 0-66 ns 전체를 맞추고 Time Units → ns를 선택한다. 신호 이름이 잘리면 이름 열 경계를 오른쪽으로 넓힌다.

파형 읽기: 저장 후 전달

15 ns 에지에서 stored가 A가 되지만 value는 0이다. load는 저장만 수행하기 때문이다.

25 ns에는 data_in0이 이미 30이다. 그래도 value는 저장해 둔 A가 된다.

이 구간이 입력을 직접 전달하는 회로와 레지스터 사이 전달 회로를 구분한다.

파형 읽기: 동시에 실행

35 ns 에지에서 stored는 3으로 바뀌고 value는 이전 stored인 A를 유지한다.

45 ns 에지에서 value가 3이 된다. 소스 코드의 위아래 순서로 즉시 전달되는 것으로 해석하지 않는다.

55 ns에는 두 제어가 0이므로 입력 F를 무시하고 유지한다. 65 ns에는 rst가 두 제어보다 우선해 둘 다 0이 된다.

수정 실험: 저장값 대신 입력 전달

1. 정상 로그와 wave.vcd를 보관한다.
2. register_pair.v의 value <= stored를 value <= data_in으로 바꾼다. TB는 바꾸지 않는다.
3. 저장 → 02 Simulate. 26 ns의 transfer stored not live input 검사가 실패하는지 확인한다.
4. 그 순간 기대값 A와 잘못 전달된 입력 3을 비교한다.
5. RTL을 원복·저장·재실행해 checks=7 통과를 확인한다.

실험 전 레포트

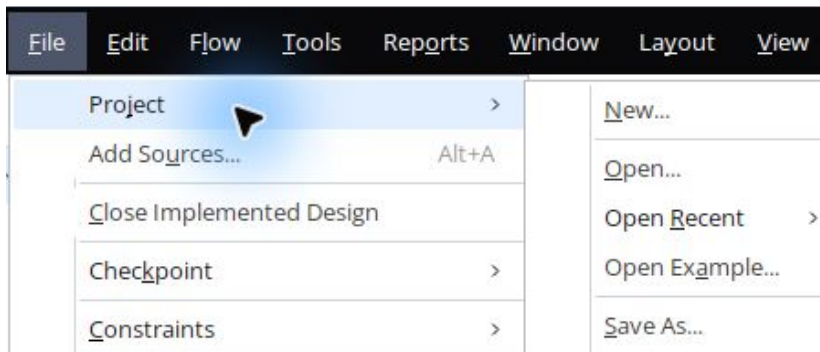
1. 제어 입력별 다음 상태 표와 동시 동작의 계산 과정을 작성한다.
2. 직접 작성한 RTL·TB·simulation.json·XDC의 역할을 설명한다.
3. 정상·변경·원복 실행 로그와 검사 시각을 비교한다.
4. VS Code 전체 파형과 25/35/45 ns 확대 파형을 넣어 해석한다.
5. 보드의 입력 순서와 예상 LED를 계산한 뒤 Vivado를 시작한다.

Vivado에서 같은 원본 연결

VS Code에서 정상·변경·원복 검사와 실험 전 레포트를 마친 뒤 Vivado를 연다.

Design top은 lab2_register, Simulation top은 tb_register_pair다.

새 프로젝트 메뉴



File → Project → New... → Next. 기존 프로젝트를 열어 둔 경우에도 같은 메뉴로 시작한다.

이름과 위치

Project name: ×

Project location: × ...

☐ Create project subdirectory

Project will be created at: C:/Users/UOS/Documents/ece2_26_2/tmp/lab2-students-0913/lab2_03_register/vivado

Project name: lab2_register. Project location: 자신의 lab2_03_register/vivado. Create project subdirectory 해제 → Next.

RTL 프로젝트



RTL Project

You will be able to add sources, create block designs in IP Integrator, generate IP, run RTL



Do not specify sources at this time



Project is an extensible Vitis platform

RTL Project와 Do not specify sources at this time 선택 → Next.

정확한 부품

Parts Boards



Q xc7s75fgga484-1



(3 matches)

Part	I/O Pin Count	Available IOBs	LUT Elements	FlipFlops
xc7s75fgga484-1	484	338	48000	96000
xc7s75fgga484-1IL	484	338	48000	96000
xc7s75fgga484-1Q	484	338	48000	96000

Parts에서 xc7s75fgga484-1 검색 → 같은 행 선택 → Next. fgga484 패키지와 -1 등급도 확인한다.

생성 요약

New Project Summary

- A new RTL project named 'lab2_register' will be created.
- The default part and product family for the new project:
Default Part: xc7s75fgga484-1
Family: Spartan-7
Package: fgga484
Speed Grade: -1

이름·위치·부품이 맞으면 Finish. 기존 프로젝트를 닫을지 묻는 창은 저장 상태 확인 후 Yes.

설계 소스 추가

- ☐ Add or create constraints
- ☒ Add or create design sources
- ☐ Add or create simulation sources

Add Sources → Add or create design sources → Next → Add Files.

RTL 세 파일 선택



src의 register_pair.v·input_frontend.v·lab2_register.v 선택 → OK.

원본 RTL 연결

	Index	Name	Library	Version	Location
●	1	input_frontend.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lab...
●	2	lab2_register.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lab...
●	3	register_pair.v	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lab...

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories

세 파일을 확인한다. Copy sources into project 해제 → Finish. VS Code 에서 수정한 원본을 계속 참조한다.

테스트벤치 추가

- ☐ Add or create constraints
- ☐ Add or create design sources
- ☒ Add or create simulation sources

Add Sources → Add or create simulation sources → Next → Add Files →
sim/tb_register_pair.sv.

TB 등록 확인

Specify simulation set: sim_1

+ - ↑ ↓					
	Index	Name	Library	Version	Location
	1	tb_register_pair.sv	xil_defaultlib	N/A	C:/Users/UOS/Documents/ece2_26_2_2/tmp/lal

- ☒ Scan and add RTL include files into project
- ☐ Copy sources into project
- ☐ Add sources from subdirectories
- ☒ Include all design sources for simulation

sim_1과 Include all design sources for simulation 유지. Copy sources into project 해제 → Finish.

XDC 추가

- ☒ Add or create constraints
- ☐ Add or create design sources
- ☐ Add or create simulation sources

Add Sources → Add or create constraints → Next → Add Files → constraints/lab2_register.xdc.

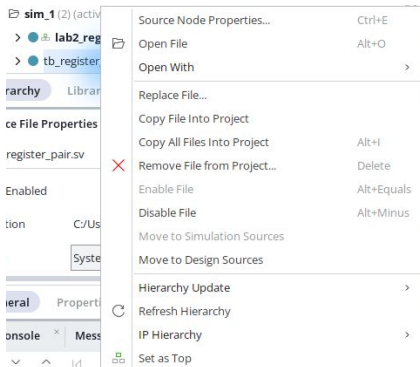
XDC 원본 연결

+ - ↑ ↓	
Constraint File	Location
lab2_register.xdc	C:\Users\UOS\Documents\ece2_26_2_2\tmp\lab2-students-0913\lab2_03_register\constraints

☐ Copy constraints files into project

Copy constraints files into project 해제 → Finish.

TB를 Top으로 설정



Sources → Simulation Sources → sim_1. tb_register_pair 우클릭 → Set as Top.

서로 다른 두 Top

Simulation Sources (2)

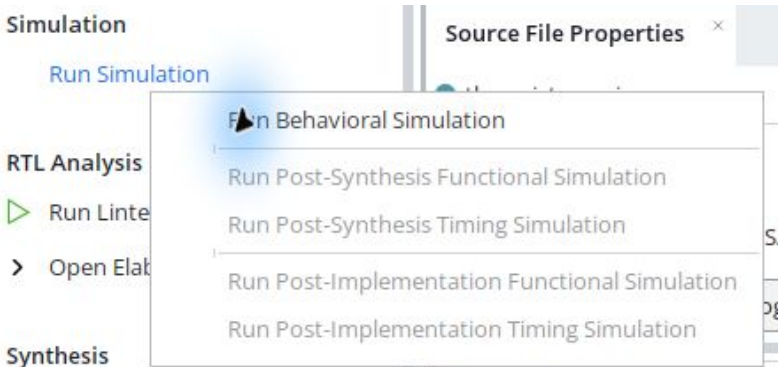
- sim_1 (2) (active)
 - tb_register_pair (tb_register_pair.sv) (1)
 - lab2_register (lab2_register.v) (2)

HierarchyLibrariesCompile Order

Project name	lab2_register
Project location	C:/Users/UOS/Documents/ece2_2
Product family	Spartan-7
Project part	xc7s75fpga484-1
Top module name	lab2_register

TB 이름이 굵게 표시된다. Project Summary의 설계 Top은 lab2_register로 유지되어야 한다.

동작 시뮬레이션



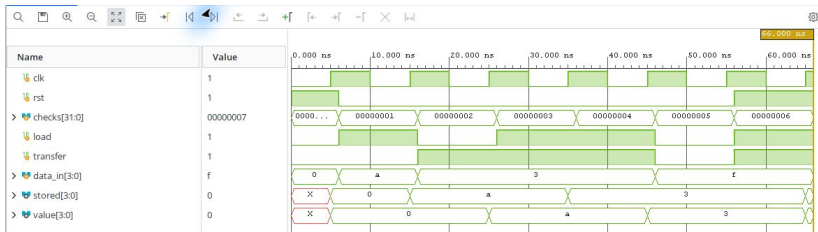
Run Simulation → Run Behavioral Simulation. 컴파일과 elaboration이 끝날 때까지 기다린다.

XSim에서도 7개 통과

```
LAB2_PASS register_pair checks=7  
$finish called at time : 66 ns : File
```

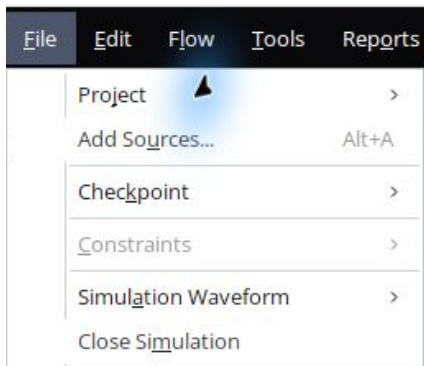
Tcl Console에서 LAB2_PASS register_pair checks=7과 66 ns 종료를 확인한다. 로그가 숨으면 조금 위로 스크롤한다.

XSim 전체 파형



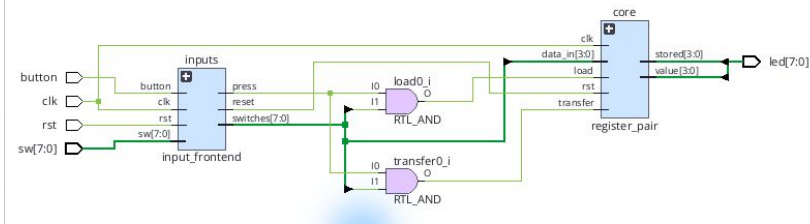
Untitled 1 탭을 더블클릭해 넓히고 Zoom Fit을 누른다. stored와 value의 변경 시각을 VS Code 파형과 대조한다.

시뮬레이션 종료



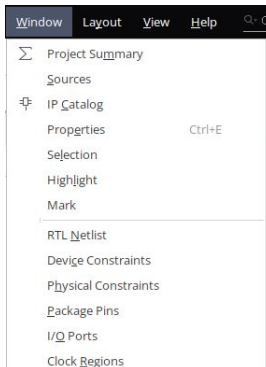
File → Close Simulation → OK. 레포트용 로그와 파형을 먼저 보관한다.

RTL 연결 확인



RTL Analysis → Open Elaborated Design → OK. 입력 처리 회로의 press와 switches가 load·transfer를 만들고 주 클록이 레지스터로 들어가는지 확인한다.

핀 표 열기



Window → I/O Ports. 아래 I/O Ports 탭을 더블클릭해 표를 뽐낸다.

LED와 스위치의 핀

Name	Direction	Neg Diff Pair	Package Pin	Fixed	Bank	I/O Std
▼ All ports (19)						
▼ led (8)	OUT			✓	35	LVC MOS33* ▼
led[7]	OUT		L4	✓	35	LVC MOS33* ▼
led[6]	OUT		M4	✓	35	LVC MOS33* ▼
led[5]	OUT		M2	✓	35	LVC MOS33* ▼
led[4]	OUT		N7	✓	35	LVC MOS33* ▼
led[3]	OUT		M7	✓	35	LVC MOS33* ▼
led[2]	OUT		M3	✓	35	LVC MOS33* ▼
led[1]	OUT		M1	✓	35	LVC MOS33* ▼
led[0]	OUT		N5	✓	35	LVC MOS33* ▼
▼ sw (8)	IN			✓	34	LVC MOS33* ▼
sw[7]	IN		Y1	✓	34	LVC MOS33* ▼
sw[6]	IN		W3	✓	34	LVC MOS33* ▼
sw[5]	IN		U2	✓	34	LVC MOS33* ▼
sw[4]	IN		T1	✓	34	LVC MOS33* ▼
sw[3]	IN		W4	✓	34	LVC MOS33* ▼
sw[2]	IN		W1	✓	34	LVC MOS33* ▼
sw[1]	IN		V4	✓	34	LVC MOS33* ▼
sw[0]	IN		U4	✓	34	LVC MOS33* ▼

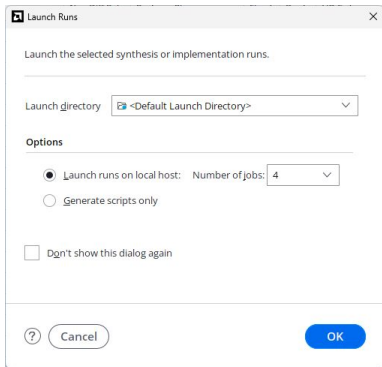
All ports 우클릭 → Expand Selection. 비트별 Package Pin·I/O Std를 직접 쓴 XDC와 대조한다.

클록과 버튼 핀

Scalar ports (3)								
button	IN		N8	✓	✓	35	LVC MOS33*	✓
clk	IN		B6	✓	✓	36	LVC MOS33*	✓
rst	IN		K4	✓	✓	35	LVC MOS33*	✓

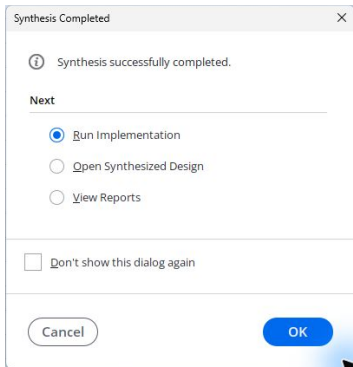
Scalar ports를 펼친다. button=N8, clk=B6, rst=K4와 LVC MOS33을 확인한다.

합성 실행



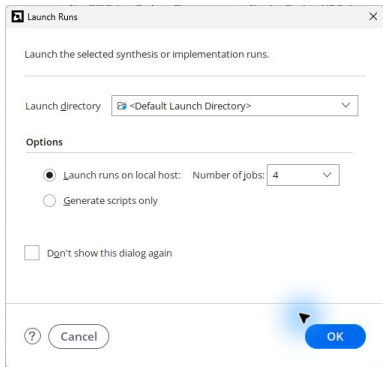
Flow Navigator → Run Synthesis. Launch runs on local host와 jobs 확인 → OK. 이 실행은 jobs=4다.

합성 완료



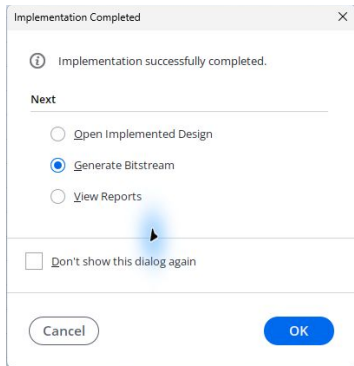
Synthesis successfully completed 확인 → Run Implementation → OK.

구현 실행



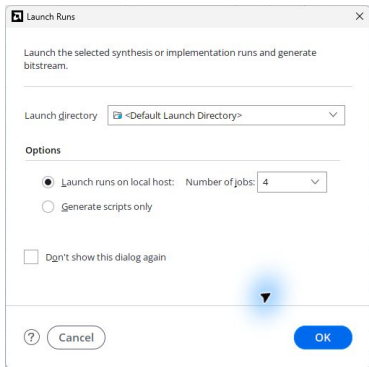
Launch runs on local host와 jobs 확인 → OK. 배치배선 중에는 소스와 XDC를 수정하지 않는다.

구현 완료



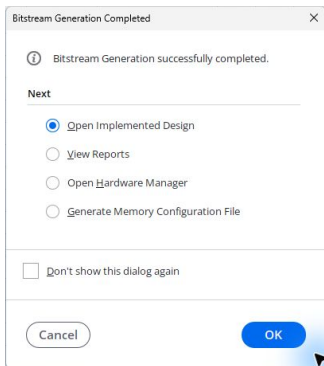
Implementation successfully completed 확인 → Generate Bitstream → OK.

bit 생성 실행



Launch runs on local host와 jobs 확인 → OK. write_bitstream이 완료될 때까지 기다린다.

bit 생성 완료



Bitstream Generation successfully completed 확인 → Open Implemented Design → OK. Elaborated Design을 닫을지 묻으면 Yes.

타이밍 결과 확인

Design Timing Summary

Setup		Hold	
Worst Negative Slack (WNS)	999997.812 ns	Worst Hold Slack (WHS)	0.119 ns
Total Negative Slack (TNS)	0.000 ns	Total Hold Slack (THS)	0.000 ns
Number of Failing Endpoints	0	Number of Failing Endpoints	0
Total Number of Endpoints	72	Total Number of Endpoints	72

아래 Timing → Design Timing Summary에서 Setup·Hold의 slack과 Failing Endpoints를 확인한다.

생성 파일과 검증 범위

생성 파일: vivado/lab2_register.runs/impl_1/lab2_register.bit.

이 교안의 실제 실행은 같은 학생 폴더의 Icarus·XSim 검사와 Vivado 합성·구현·bit 생성까지 확인했다.

다음 장치 기록과 촬영은 학생이 실습 장비에서 수행한다. bit 생성 화면은 실제 보드 동작을 촬영한 결과가 아니다.

타이밍과 남은 경고

WNS=999997.812 ns, WHS=0.119 ns. Setup·Hold의 실패 endpoint는 각각 0이며 지정한 내부 타이밍 제약을 만족했다. 보드 주기는 1,000,000 ns다.

TIMING-18: LED 출력 8개의 외부 지연 제약이 없다. 외부 동기 수신기의 조건을 정하지 않았으므로 내부 타이밍 통과를 모든 입출력 타이밍 보장으로 해석하지 않는다.

CFGBVS-1: 설정 बैं크의 전압 속성이 지정되지 않았다. 보드 회로도·설정 전압을 확인한 뒤 값을 정하며 임의의 전압을 입력해 경고만 없애지 않는다.

장치에 기록

1. 보드 전원·JTAG·주 클록 1 kHz를 확인한다.
2. Open Hardware Manager → Open Target → Auto Connect.
3. 장치 우클릭 → Program Device → 방금 생성한 lab2_register.bit → Program.
4. K4로 초기화한다. 아래 표대로 DIP 입력을 정하고 N8 버튼을 누른 뒤 놓는다.

보드 확인 순서

sw[7:0] (16진수)	N8 한 번 실행 후 LED[7:0]
A1	A0
32	AA
33	3A
33, 다시 누름	33
F0	33 유지
K4 초기화	00

스위치를 먼저 안정시킨 다음 버튼을 누른다. 값과 제어 비트를 구분해 이 결과를 계산한다.

실험 후 레포트

1. 동일 TB의 XSim PASS·66 ns 종료·파형을 Icarus와 비교한다.
2. 부품·핀·클록·합성·구현·bit 파일과 타이밍·경고를 기록한다.
3. 실제 Program Device 화면과 보드 전체·LED 사진을 넣는다.
4. 동시 load/transfer를 두 번 실행하는 장면과 초기화 영상을 촬영한다.
5. 소스·전후 레포트·사진·영상 링크를 GitHub README에서 연결한다.